

Machine Learning A (2026)

Home Assignment 5

Ludovico Maria Spitaleri, DMH249

Contents

1 On the Role of Dependence (8 points) [Sadegh, W5 Monday]	1
2 On Confidence Intervals (12 points) [Sadegh, W5 Monday]	1
2.1 Part 1	1
2.2 Part 2	1
3 Loss Range Correction in Generalization Bounds (10 points) [Sadegh, W5 Monday]	1
3.1 (i)	1
3.2 (ii)	1
4 Convolutional Neural Networks (70 points) [Christian, W5 Friday]	1
4.1 Sobel filter (24 points) [Christian, W5 Friday]	1
4.2 Convolutional neural networks (24 points) [Christian, W5 Friday]	2
4.3 Augmentation (22 points) [Christian, W5 Friday]	3

1 On the Role of Dependence (8 points) [Sadegh, W5 Monday]

2 On Confidence Intervals (12 points) [Sadegh, W5 Monday]

2.1 Part 1

2.2 Part 2

3 Loss Range Correction in Generalization Bounds (10 points) [Sadegh, W5 Monday]

3.1 (i)

3.2 (ii)

4 Convolutional Neural Networks (70 points) [Christian, W5 Friday]

4.1 Sobel filter (24 points) [Christian, W5 Friday]

The Sobel filter is implemented with a single `nn.Conv2d` with one input and two output channels. The magnitude G is obtained by squaring, summing over the channel axis and taking the square root.

```

Kx = torch.tensor([[1., 0., -1.],
                   [2., 0., -2.],
                   [1., 0., -1.]])
Ky = torch.tensor([[ 1.,  2.,  1.],
                   [ 0.,  0.,  0.],
                   [-1., -2., -1.]])
K = torch.stack([Kx, Ky]).unsqueeze(1)
K = torch.flip(K, dims=[-2, -1])

conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
conv.weight = nn.Parameter(K, requires_grad=False)

g = conv(x)
G = torch.sqrt(torch.sum(torch.square(g), dim=1))
Gx, Gy = g[0, 0], g[0, 1]

```

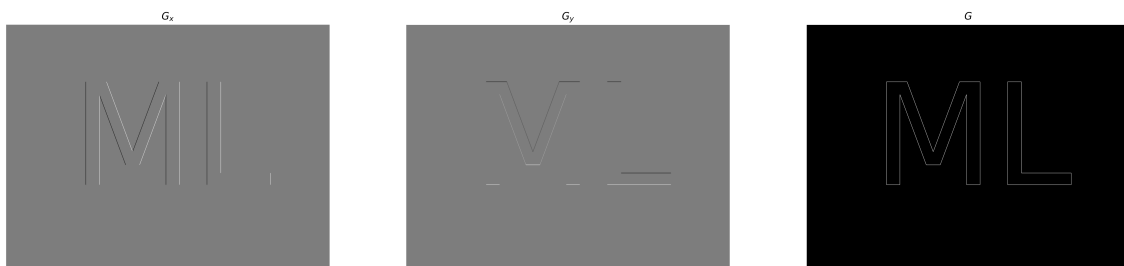


Figure 1: The Sobel feature maps G_x , G_y and G .

`nn.Conv2d` does not flip, it computes the cross-correlation $[K \star I]_{ij} = \sum_{u,v} K_{uv} I_{i+u, j+v}$. Using G_x, G_y unchanged would therefore compute the correlation with those matrices, which equals the convolution with the *flipped* matrices, so G_x and G_y would come out with the opposite sign. To implement the convolution the kernels must be flipped before assigning them as weights.

The final map G is unaffected by this choice, since it only depends on G_x^2 and G_y^2 and a flip of these antisymmetric kernels only changes the sign of G_x and G_y .

4.2 Convolutional neural networks (24 points) [Christian, W5 Friday]

```

class Net(nn.Module):
    def __init__(self, img_size=28):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=5) # 28x28 -> 24x24
        self.pool1 = nn.MaxPool2d(2) # 24x24 -> 12x12
        self.conv2 = nn.Conv2d(64, 64, kernel_size=5) # 12x12 -> 8x8
        self.pool2 = nn.MaxPool2d(2) # 8x8 -> 4x4
        self.fc2 = nn.Linear(64 * 4 * 4, 43) # 1024 -> 43

    def forward(self, x):
        x = self.pool1(F.elu(self.conv1(x)))
        x = self.pool2(F.elu(self.conv2(x)))

```

```
x = torch.flatten(x, 1)
return self.fc2(x)
```

The input crops are 28×28 . Each 5×5 convolution without padding removes 4 pixels per dimension, and each pooling halves the size: $28 \rightarrow 24 \rightarrow 12 \rightarrow 8 \rightarrow 4$. This gives $64 \cdot 4 \cdot 4 = 1024$ inputs to the fully-connected layer, matching `in_features=1024`. The output layer returns logits (no softmax), since `CrossEntropyLoss` applies it internally.

4.3 Augmentation (22 points) [Christian, W5 Friday]

Transformations in the original code. During training, each image is (1) resized to 32×32 , (2) randomly rotated by an angle in $[-5^\circ, 5^\circ]$ (`RandomAffine((-5, 5))`), (3) randomly cropped to 28×28 (a random translation of up to 4 pixels), (4) colour-jittered (brightness factor in $[0.2, 1.8]$, contrast factor in $[0.6, 1.4]$), and (5) for some classes (11, 12, 13, 17, 18, 26, 30, 35), horizontally flipped with probability 0.5. At test time only the resize and a deterministic centre crop are applied.

Why is the flip conditioned on the label? Augmentation must not change the label. Signs of classes 11, 12, 13, 17, 18, 26, 30 and 35 (right-of-way, priority road, yield, no entry, general caution, traffic signals, ice/snow, ahead only) are (approximately) mirror-symmetric, so a flipped image still shows the same class. For other classes a flip changes the meaning: a “turn left ahead” sign becomes a “turn right ahead” sign, and flipping a speed limit produces mirrored digits that never occur in real data. Flipping those would introduce label noise or unrealistic inputs.

Additional transformations. I added two transformations to `__getitem__` (after the original ones):

```
# in the training branch, before the random crop
image = transforms.RandomPerspective(distortion_scale=0.15, p=0.5)
(image)
...
# after ToTensor (RandomErasing operates on tensors)
image = transforms.RandomErasing(p=0.25, scale=(0.02, 0.08), ratio=(0.3,
3.3), value=0)(image)
```

- **Random perspective:** Cars see signs from different positions and angles, so signs appear skewed in the images. The existing affine transformation only covers small rotations and translations, while a perspective warp approximates changes in viewpoint. Mild distortion (0.15) keeps the sign recognisable.
- **Random erasing:** The task description lists partial occlusions as a source of variation (trees, dirt, stickers). Erasing a small random patch (2–8% of the image, 25% of the time) forces the network not to rely on a single small region. The patch is kept small so that it rarely removes all information needed for the class.

Both transformations are applied only to the training data; the test pipeline is unchanged.